

Resilience Demo

Code & Concept Guide

What each part does, why it exists, the exact code that implements it, and what it looks like running

Contents

1. Overview & Architecture

2. Project Structure

3. Logging

4. Retry

5. Timeout

6. Fallback

7. Circuit Breaker

8. Spring Boot Actuator

9. Micrometer & Custom Metrics

10. Resilience4j → Micrometer Binding

11. Prometheus

12. Grafana

13. Complete Request Flow

14. Running the Project

1. Overview & Architecture

Resilience Demo is two Spring Boot services plus a small browser UI. Order Service receives an order, calls Payment Service over HTTP, and wraps that call in Retry, a Circuit Breaker, a Timeout, and a Fallback. Both services log what they do and publish metrics that Prometheus scrapes and Grafana visualizes.

```
Browser UI
| POST /orders
v
Order Service :8082
| OrderController -> OrderService
| Retry -> Circuit Breaker -> Timeout -> RestClient
v
Payment Service :8081
PaymentController - SUCCESS / HTTP 500 / ~5s delay
```

Observability, independently of the flow above:

```
Spring Boot -> Actuator + Micrometer -> /actuator/prometheus -> Prometheus :9092 -> Grafana :3030
```

Logging, independently of both:

```
Application code -> SLF4J (API) -> Logback (implementation) -> Console
```

Stack: Java 21, Spring Boot 4.1.2-SNAPSHOT, `spring-boot-starter-webmvc`, Spring's `RestClient`, Resilience4j 2.2.0 (Retry, TimeLimiter, CircuitBreaker, Micrometer binding — Order Service only), Actuator + Micrometer + `micrometer-registry-prometheus` (both services), Prometheus + Grafana (Docker), plain HTML/CSS/JS for the UI.

2. Project Structure

```
order-service/           Spring Boot app, port 8082 (the caller)
  src/main/java/com/resilience/demo/order/
    OrderServiceApplication.java
    controller/ OrderController, OrderRequest, OrderResponse, CircuitBreakerStatus
    service/      OrderService (Retry/CircuitBreaker/Timeout live here), PaymentResponse
  src/main/resources/application.properties

payment-service/         Spring Boot app, port 8081 (the callee)
  src/main/java/com/resilience/demo/payment/
    PaymentServiceApplication.java
    controller/   PaymentController, PaymentResponse
  src/main/resources/application.properties

ui/                       Static HTML/CSS/JS, no build step
prometheus/prometheus.yml Scrape config for both services
grafana/provisioning/     Auto-provisioned datasource + dashboard
docker-compose.yml        Prometheus + Grafana containers only
```

3. Logging

Concept

Logging is the application writing timestamped statements about what it's doing, so you can see what happened without a debugger. This project uses three levels deliberately: **INFO** for a normal expected step (request received, calling downstream, response received), **WARN** for something recoverable (a retry about to happen, a timeout, a circuit breaker rejecting a call or changing state), **ERROR** for a final failure (a simulated payment failure, the fallback being returned).

Imports used

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
```

`Logger` is the SLF4J interface every `log.info/warn/error(...)` call is made on. `LoggerFactory` is the factory class that creates a `Logger` instance. SLF4J is the logging *API* the code is written against; Logback is the logging *implementation* Spring Boot wires in underneath by default — no Logback dependency or config file was added by hand, it just prints to the console.

The code

```
private static final Logger log = LoggerFactory.getLogger(OrderService.class);
```

Part	Explanation
<code>private static final</code>	One logger instance shared by every instance of the class, never reassigned
<code>LoggerFactory.getLogger(OrderService.class)</code>	Creates a logger named after this class (<code>com.resilience.demo.order.service.OrderService</code>) — that name is what <code>logging.level.com.resilience.demo.order=INFO</code> in <code>application.properties</code> filters on

```
log.info("Order request received: orderId={}, paymentType={}", request.orderId(), request.paymentType());
```

Method	Purpose	Used for
<code>log.info(String, Object...)</code>	Record a normal event; <code>{}</code> placeholders are filled by the trailing arguments, in order	Request received, payment attempt, response received, order completed
<code>log.warn(String, Object...)</code>	Record something recoverable	Retrying, call timed out, call rejected by an open circuit breaker, circuit breaker state changes to OPEN/HALF_OPEN
<code>log.error(String, Object, Object...)</code>	Record a final failure	Simulated payment failure
<code>log.error(String, Object, Object, Throwable)</code>	Same, but the last argument is the actual exception — SLF4J detects a trailing <code>Throwable</code> and prints its full stack trace under the message instead of substituting it into <code>{}</code>	<code>OrderService</code> 's fallback catch blocks

```
log.error("Payment Service call failed: orderId={}, httpStatus={}", request.orderId(),
    e.getStatusCode().value(), e); // e is the exception -> full stack trace printed below
```

Example — actual console output for one Normal order

```
INFO OrderService : Order request received: orderId=1001, paymentType=NORMAL
INFO OrderService : Payment attempt: orderId=1001, attempt=1
INFO PaymentController : Payment request received: orderId=1001
INFO PaymentController : Payment successful: orderId=1001
INFO OrderService : Payment response received: orderId=1001, status=SUCCESS
INFO OrderService : Order completed: orderId=1001
```

4. Retry

Concept

Retry means: if a call fails in a way that looks temporary, try again a small, bounded number of times before giving up. Payment Service's `/payments/failure/{orderId}` endpoint always returns HTTP 500 — the kind of failure Retry exists to absorb.

Imports used

```
import io.github.resilience4j.retry.Retry;
import io.github.resilience4j.retry.RetryConfig;
import org.springframework.web.client.RestClientResponseException;
import java.time.Duration;
```

The code

```
this.paymentRetry = Retry.of("paymentRetry", RetryConfig.custom()
    .maxAttempts(retryMaxAttempts) // 3 = 1 initial call + 2 retries
    .waitDuration(Duration.ofMillis(retryWaitMs)) // 500ms between attempts
    .retryOnException(t -> t instanceof RestClientResponseException rex
        && rex.getStatusCode().is5xxServerError())
    .build());
```

Class	Method	What it does	Value used
RetryConfig	custom()	Starts a fluent builder for a retry policy	—
Builder	maxAttempts(int)	Total attempts, including the first — bounds how long a failing call can drag on	3
Builder	waitDuration(Duration)	Fixed delay between one failed attempt and the next	500ms
Builder	retryOnException(Predicate<Throwable>)	Decides which exceptions are worth retrying at all — here, only a real 5xx server error	RestClientResponseException with is5xxServerError()
Builder	build()	Finalizes the immutable config	—
Retry	Retry.of(String, RetryConfig)	Creates a named, reusable <code>Retry</code> instance from the config, held as a field	name "paymentRetry"
Retry (instance)	executeCallable(Callable<T>)	Runs the lambda; if it throws a retryable exception, waits and runs it again — up to <code>maxAttempts</code> times, blocking the calling thread between attempts	—

```
private PaymentResponse callWithResilience(...) throws Exception {
    return paymentRetry.executeCallable(() -> {
        int attempt = attempts.incrementAndGet();
        if (attempt == 1) log.info("Payment attempt: orderId={}, attempt={}", orderId, attempt);
        else log.warn("Retrying payment: orderId={}, attempt={}", orderId, attempt);
        // Circuit Breaker + Timeout guarded call happens inside here (Sections 5 and 7)
    });
}
```

This is the outermost layer of the resilience chain: each attempt is its own Circuit-Breaker call and its own Timeout-guarded call, so if the endpoint fails 3 times, the Circuit Breaker records 3 separate outcomes.

Example — actual console output and response for Failure

```
INFO OrderService : Payment attempt: orderId=2002, attempt=1
WARN OrderService : Retrying payment: orderId=2002, attempt=2
WARN OrderService : Retrying payment: orderId=2002, attempt=3
ERROR OrderService : Fallback executed: orderId=2002, reason=InternalServerError

Response:
{"orderId":"2002","paymentStatus":"FALLBACK","paymentHttpStatus":0,"responseTimeMs":1030,
 "message":"Payment service temporarily unavailable","mechanism":"RETRY","attempts":3,
 "circuitState":"CLOSED","paymentServiceCalled":true}
```

Three attempts, ~500ms apart (~1030ms total), `mechanism: "RETRY"`, `attempts: 3` — exactly matching the configured policy.

5. Timeout

Concept

Timeout means: don't wait forever for a response. Payment Service's `/payments/slow/{orderId}` sleeps ~5 seconds; without a timeout, Order Service would simply hang for the full 5 seconds.

Why `CompletableFuture` is needed

Resilience4j's `TimeLimiter` races a *future* against a clock — `Future.get(timeout, unit)` internally. Spring's `RestClient` call is blocking, not future-based, so it's wrapped in `CompletableFuture.supplyAsync(...)`, which runs it on another thread and immediately hands back a future the `TimeLimiter` can wait on.

Imports used

```
import io.github.resilience4j.timelimiter.TimeLimiter;
import io.github.resilience4j.timelimiter.TimeLimiterConfig;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeoutException;
```

The code

```
private final ExecutorService paymentExecutor = Executors.newVirtualThreadPerTaskExecutor();

this.paymentTimeout = TimeLimiter.of("paymentTimeout", TimeLimiterConfig.custom()
    .timeoutDuration(Duration.ofMillis(timeoutMs)) // 2000ms
    .build());
```

Class	Method	What it does	Value used
<code>Executors</code>	<code>newVirtualThreadPerTaskExecutor()</code>	Creates an executor that runs each submitted task on a new lightweight virtual thread (Java 21)	—
<code>TimeLimiterConfig</code>	<code>custom()</code>	Starts a fluent builder	—
Builder	<code>timeoutDuration(Duration)</code>	Maximum time to wait for the future to complete	2000ms — deliberately below the 5s slow-endpoint delay so it actually triggers
Builder	<code>build()</code>	Finalizes the config	—
<code>TimeLimiter</code>	<code>TimeLimiter.of(String, TimeLimiterConfig)</code>	Creates the named, reusable <code>TimeLimiter</code>	name "paymentTimeout"
<code>CompletableFuture</code>	<code>supplyAsync(Supplier, Executor)</code>	Runs the blocking call on <code>paymentExecutor</code> , returns immediately with a future	—
<code>TimeLimiter</code> (instance)	<code>executeFutureSupplier(Supplier<Future<T>>)</code>	Calls the supplier to get a future, then blocks up to the timeout waiting on it; on timeout, cancels the future and throws <code>TimeoutException</code>	—

```
CompletableFuture<PaymentResponse> future = CompletableFuture.supplyAsync(
    () -> paymentServiceClient.get().uri(path).retrieve().body(PaymentResponse.class),
    paymentExecutor);

try {
    return paymentTimeout.executeFutureSupplier(() -> future);
} catch (TimeoutException e) {
    log.warn("Payment call timed out: orderId={}, attempt={}", orderId, attempt);
    throw e;
}
```

A `TimeoutException` does not match Retry's `retryOnException` predicate (Section 4), so a timeout is never retried — it goes straight to fallback after exactly 1 attempt.

Example — actual console output and response for Slow

```
INFO OrderService : Payment attempt: orderId=2003, attempt=1
WARN OrderService : Payment call timed out: orderId=2003, attempt=1
ERROR OrderService : Fallback executed: orderId=2003, reason=timeout
```

Response:

```
{"orderId":"2003","paymentStatus":"FALLBACK","paymentHttpStatus":0,"responseTimeMs":2003,
  "message":"Payment service temporarily unavailable","mechanism":"TIMEOUT","attempts":1,
  "circuitState":"CLOSED","paymentServiceCalled":true}
```

Cut off at 2003ms — matching the configured 2000ms timeout — with only 1 attempt, and `paymentServiceCalled: true` because the call was actually sent (it just never returned in time).

6. Fallback

Concept

Fallback is what gets returned once Order Service has given up on the real call — retries exhausted, timed out, or rejected by an open circuit breaker — instead of propagating an error. It is plain `try/catch` in `OrderService.createOrder`, not an annotation or a separate class, so the whole resilience story reads top-to-bottom in one method.

The code

```
try {
    PaymentResponse payment = callWithResilience(request.orderId(), path, attempts, paymentCalled);
    // ... success path: mechanism = "NONE" or "RETRY" ...
} catch (CallNotPermittedException e) {
    log.error("Fallback executed: orderId={}, reason=circuit-breaker-open", request.orderId());
    ordersFallback.increment();
    return new OrderResponse(request.orderId(), "FALLBACK", 0, responseTime,
        "Payment service temporarily unavailable", "CIRCUIT_BREAKER", attempts.get(),
        currentCircuitState(), paymentCalled.get());
} catch (TimeoutException e) {
    log.error("Fallback executed: orderId={}, reason=timeout", request.orderId());
    ordersFallback.increment();
    return new OrderResponse(request.orderId(), "FALLBACK", 0, responseTime,
        "Payment service temporarily unavailable", "TIMEOUT", attempts.get(),
        currentCircuitState(), paymentCalled.get());
} catch (Exception e) {
    String mechanism = attempts.get() > 1 ? "RETRY" : "FALLBACK";
    log.error("Fallback executed: orderId={}, reason={}", request.orderId(), e.getClass().getSimpleName());
    ordersFallback.increment();
    return new OrderResponse(request.orderId(), "FALLBACK", 0, responseTime,
        "Payment service temporarily unavailable", mechanism, attempts.get(),
        currentCircuitState(), paymentCalled.get());
}
```

Every catch block does the same three things — logs an ERROR with a specific `reason`, increments the `ordersFallback` counter, and returns an `OrderResponse` with `paymentStatus="FALLBACK"` — differing only in the `mechanism` label, so the response tells you *why* it fell back, not just that it did.

Example — how the UI displays each fallback

Failure ->	Payment Status: FALLBACK	Mechanism: RETRY	Attempts: 3
Slow ->	Payment Status: FALLBACK	Mechanism: TIMEOUT	Attempts: 1
Open CB ->	Payment Status: FALLBACK	Mechanism: CIRCUIT_BREAKER	Attempts: 1

In the UI (`script.js`), any `paymentStatus === "FALLBACK"` renders an amber "⚠" banner, visually distinct from a green success.

7. Circuit Breaker

Concept

A Circuit Breaker watches outcomes across many calls, and once failures cross a threshold, it stops calling the dependency entirely for a while — failing fast instead of retrying into a service that's already struggling. Retry acts within one request; the Circuit Breaker protects the dependency across all requests, and can even stop calling it mid-request.

The three states

```
CLOSED --(failure threshold reached)--> OPEN --(wait duration elapses,
automatic)--> HALF_OPEN --(test calls succeed)--> CLOSED
                                HALF_OPEN --(test call fails)--> OPEN
```

State	What happens to a call
CLOSED	Goes through normally; the outcome is recorded
OPEN	Rejected immediately with <code>CallNotPermittedException</code> — Payment Service is never called
HALF_OPEN	A small number of test calls decide CLOSED or OPEN next

Imports used

```
import io.github.resilience4j.circuitbreaker.CallNotPermittedException;
import io.github.resilience4j.circuitbreaker.CircuitBreaker;
import io.github.resilience4j.circuitbreaker.CircuitBreakerConfig;
import io.github.resilience4j.circuitbreaker.CircuitBreakerRegistry;
import io.github.resilience4j.micrometer.tagged.TaggedCircuitBreakerMetrics;
```

The code

```
CircuitBreakerRegistry circuitBreakerRegistry = CircuitBreakerRegistry.of(CircuitBreakerConfig.custom()
    .slidingWindowType(CircuitBreakerConfig.SlidingWindowType.COUNT_BASED)
    .slidingWindowSize(5) // remembers the last 5 calls
    .minimumNumberOfCalls(5) // needs 5 calls before evaluating
    .failureRateThreshold(50) // >=50% failing trips it
    .waitDurationInOpenState(Duration.ofMillis(5000))
    .permittedNumberOfCallsInHalfOpenState(2)
    .automaticTransitionFromOpenToHalfOpenEnabled(true) // flips on its own, on a timer
    .build());
this.paymentCircuitBreaker = circuitBreakerRegistry.circuitBreaker("paymentCircuitBreaker");
```

Class	Method	What it does	Value used
CircuitBreakerConfig	custom()	Starts a fluent builder	—
Builder	slidingWindowType(...)	Count-based window: remembers the last N calls, not a time window	COUNT_BASED
Builder	slidingWindowSize(int)	How many recent calls the window remembers	5
Builder	minimumNumberOfCalls(int)	Calls needed before the failure rate is even evaluated	5
Builder	failureRateThreshold(float)	Failure percentage that trips OPEN	50
Builder	waitDurationInOpenState(Duration)	How long to stay OPEN before a HALF_OPEN test is allowed	5000ms
Builder	permittedNumberOfCallsInHalfOpenState(int)	How many test calls are allowed through in HALF_OPEN	2
Builder	automaticTransitionFromOpenToHalfOpenEnabled(boolean)	Lets OPEN→HALF_OPEN happen on a background timer, not only on the next call	true
CircuitBreakerRegistry	CircuitBreakerRegistry.of(CircuitBreakerConfig)	Creates a registry that hands out circuit breakers built from this config	—
CircuitBreakerRegistry (instance)	circuitBreaker(String)	Gets (creating if absent) the named circuit breaker — using the registry here (instead of CircuitBreaker.of(...) directly) is what lets the Micrometer binder in Section 10 discover it	name "paymentCircuitBreaker"
CircuitBreaker (instance)	executeCallable(Callable<T>)	Runs the callable only if the breaker permits it (CLOSED, or a free HALF_OPEN test slot); otherwise throws CallNotPermittedException without running it at all, and records the outcome	—
CircuitBreaker (instance)	getState() / getName()	Reads the current state / configured name	used by getCircuitBreakerStatus()
CircuitBreaker (instance)	getEventPublisher().onStateTransition(...)	Registers a callback fired on every state change	logs the new state

```
try {
    return paymentCircuitBreaker.executeCallable() -> {
        paymentCalled.set(true);
        // ... CompletableFuture + Timeout guarded RestClient call (Section 5) ...
    };
} catch (CallNotPermittedException e) {
    log.warn("Payment call rejected: circuit breaker OPEN, orderId={}, attempt={}", orderId, attempt);
    throw e;
}
```

If the breaker is OPEN, the lambda never runs at all — that's why `paymentCalled.set(true)` never executes for a rejected call, and the response correctly reports `paymentServiceCalled: false`. Actual nesting order for one attempt: `Retry` → `CircuitBreaker` → `Timeout` → `RestClient` — each retry attempt is its own circuit-breaker call, so a couple of Failure requests (each retried 3 times) is enough to fill the 5-call window and trip it open.

```
paymentCircuitBreaker.getEventPublisher().onStateTransition(event -> {
    CircuitBreaker.State toState = event.getStateTransition().getToState();
    if (toState == CircuitBreaker.State.CLOSED) log.info("Circuit Breaker state: {}", toState);
    else log.warn("Circuit Breaker state: {}", toState);
});
```

Example — actual state transition, observed live

1st Failure click: 3 calls recorded, still CLOSED (below minimumNumberOfCalls=5)
2nd Failure click: attempts 1-2 fail (5th call reached, 100% failure rate) -> circuit OPENS
mid-request -> attempt 3 rejected instantly

WARN OrderService : Circuit Breaker state: OPEN

WARN OrderService : Payment call rejected: circuit breaker OPEN, orderId=3007, attempt=3

ERROR OrderService : Fallback executed: orderId=3007, reason=circuit-breaker-open

Next request (any type), sent immediately:

```
{"orderId":"3008","paymentStatus":"FALLBACK","paymentHttpStatus":0,"responseTimeMs":0,
  "message":"Payment service temporarily unavailable","mechanism":"CIRCUIT_BREAKER",
  "attempts":1,"circuitState":"OPEN","paymentServiceCalled":false}
```

5 seconds later, with no request sent at all:

WARN OrderService : Circuit Breaker state: HALF_OPEN (fired by a background timer thread)

Two successful test calls while HALF_OPEN:

INFO OrderService : Circuit Breaker state: CLOSED

`responseTimeMs: 0` and `paymentServiceCalled: false` prove Payment Service was not called at all — the rejection happened purely inside Order Service.

8. Spring Boot Actuator

Actuator exposes ready-made HTTP endpoints for inspecting a running app.

```
management.endpoints.web.exposure.include=health,metrics,prometheus
```

Endpoint	Purpose
GET /actuator/health	Is the app up?
GET /actuator/metrics	Lists every registered metric name (JVM, HTTP, custom)
GET /actuator/prometheus	Full metrics dump in Prometheus's text format — this is what Prometheus itself scrapes

Example — actual responses

```
GET /actuator/health
{"groups":["liveness","readiness"],"status":"UP"}

GET /actuator/prometheus (excerpt)
orders_processed_total 4.0
orders_successful_total 1.0
orders_fallback_total 3.0
resilience4j_circuitbreaker_state{name="paymentCircuitBreaker",state="open"} 1.0
```

9. Micrometer & Custom Metrics

Micrometer is the metrics facade Spring Boot builds on — application code records values against its API, and the registered exporter (`micrometer-registry-prometheus`) decides the storage/export format.

Imports used

```
import io.micrometer.core.instrument.Counter;
import io.micrometer.core.instrument.MeterRegistry;
```

`MeterRegistry` is auto-configured by Spring Boot once `micrometer-registry-prometheus` is on the classpath, and is obtained purely through constructor injection — no manual bean definition exists in this project.

The code

```
public OrderService(..., MeterRegistry meterRegistry) {
    this.ordersProcessed = Counter.builder("orders.processed")
        .description("Total order requests received")
        .register(meterRegistry);
    this.ordersSuccessful = Counter.builder("orders.successful")
        .description("Order requests that completed with a successful payment")
        .register(meterRegistry);
    this.ordersFallback = Counter.builder("orders.fallback")
        .description("Order requests that ended in the fallback response")
        .register(meterRegistry);
    ...
}
```

Method	Purpose
<code>Counter.builder(String name)</code>	Names the metric — Prometheus exports it with dots turned into underscores and a <code>_total</code> suffix, e.g. <code>orders.processed</code> → <code>orders_processed_total</code>
<code>.description(String)</code>	Human-readable help text shown in Prometheus/Grafana
<code>.register(MeterRegistry)</code>	Registers the counter and returns the live instance to hold onto
<code>Counter.increment()</code>	Called later, at the point the event actually occurs, to add 1

```
// PaymentController.java
public PaymentController(MeterRegistry meterRegistry) {
    this.paymentRequests = Counter.builder("payment.requests")
        .description("Total payment requests received").register(meterRegistry);
    this.paymentSuccessful = Counter.builder("payment.successful")
        .description("Payment requests that returned SUCCESS").register(meterRegistry);
    this.paymentFailed = Counter.builder("payment.failed")
        .description("Payment requests that returned FAILED").register(meterRegistry);
}
```

Service	Metric	Incremented when
Order Service	<code>orders.processed</code>	Every order request received
	<code>orders.successful</code>	Order completed with a successful payment
	<code>orders.fallback</code>	Order ended in the fallback response
Payment Service	<code>payment.requests</code>	Any of the 3 endpoints is hit
	<code>payment.successful</code>	Normal or slow endpoint returns SUCCESS
	<code>payment.failed</code>	Failure endpoint returns FAILED

Example — actual values after Normal + Failure×2 + Slow

```
orders_processed_total 4.0    orders_fallback_total 3.0
orders_successful_total 1.0   payment_requests_total 5.0
payment_successful_total 2.0   payment_failed_total 3.0
```

10. Resilience4j → Micrometer Binding

```
TaggedCircuitBreakerMetrics.ofCircuitBreakerRegistry(circuitBreakerRegistry).bindTo(meterRegistry);
```

Method	Purpose
<code>TaggedCircuitBreakerMetrics.ofCircuitBreakerRegistry(CircuitBreakerRegistry)</code>	Builds a Micrometer <code>MeterBinder</code> that watches every circuit breaker in the given registry
<code>.bindTo(MeterRegistry)</code>	Actually registers the metrics into the given registry

This single line publishes `resilience4j_circuitbreaker_state`, `resilience4j_circuitbreaker_failure_rate`, `resilience4j_circuitbreaker_calls_seconds_count`, and `resilience4j_circuitbreaker_not_permitted_calls_total` into the same Prometheus scrape as the custom counters, with no hand-written metric code.

Example — actual output while CLOSED

```
resilience4j_circuitbreaker_state{name="paymentCircuitBreaker",state="closed"} 1.0
resilience4j_circuitbreaker_state{name="paymentCircuitBreaker",state="open"} 0.0
resilience4j_circuitbreaker_state{name="paymentCircuitBreaker",state="half_open"} 0.0
resilience4j_circuitbreaker_not_permitted_calls_total{kind="not_permitted",name="paymentCircuitBreaker"} 0.0
```

11. Prometheus

Prometheus *pulls*: it periodically makes an HTTP request to each target's metrics endpoint.

```
global:
  scrape_interval: 5s
scrape_configs:
  - job_name: order-service
    metrics_path: /actuator/prometheus
    static_configs: [{ targets: ["host.docker.internal:8082"] }]
  - job_name: payment-service
    metrics_path: /actuator/prometheus
    static_configs: [{ targets: ["host.docker.internal:8081"] }]
```

Prometheus runs in Docker; both Spring apps run on the host. `host.docker.internal` is Docker Desktop's DNS name for reaching the host machine from inside a container. Every metric scraped under a job is tagged with that job's name, which is how Grafana tells the two services' identically-named `http_server_requests_seconds_count` apart.

Example — Prometheus target status

order-service	UP	http://host.docker.internal:8082/actuator/prometheus
payment-service	UP	http://host.docker.internal:8081/actuator/prometheus

12. Grafana

Grafana queries a datasource (here, Prometheus) and renders results as dashboard panels. Both the datasource and the dashboard are provisioned from files.

```
# grafana/provisioning/datasources/datasource.yml
datasources:
- name: Prometheus
  uid: prometheus
  url: http://prometheus:9090    # Compose service name, not the host port
  isDefault: true
```

`http://prometheus:9090` works because Grafana and Prometheus share a Docker Compose network and resolve each other by service name.

Panel	Query
Order Service request rate	<code>sum(rate(http_server_requests_seconds_count{job="order-service", uri="/orders"}[1m]))</code>
Payment Service request rate	<code>same, job="payment-service", grouped by (uri)</code>
Order/Payment response duration (avg)	<code>rate(..._sum[1m]) / rate(..._count[1m])</code>
Order fallback count	<code>orders_fallback_total, orders_processed_total, orders_successful_total</code>
Circuit Breaker state / failure rate	<code>resilience4j_circuitbreaker_state{name="paymentCircuitBreaker"}, resilience4j_circuitbreaker_failure_rate{...}</code>

13. Complete Request Flow

Normal

```
Retry attempt 1 -> CircuitBreaker permits (CLOSED) -> Timeout window (well under 2s)
-> RestClient GET /payments/{id} -> SUCCESS
-> mechanism=NONE, attempts=1
```

Failure

```
attempt 1 fails (HTTP 500, retryable) -> wait 500ms
attempt 2 fails -> wait 500ms
attempt 3 fails -> attempts exhausted -> catch(Exception) -> fallback
-> mechanism=RETRY, attempts=3
(each attempt also counts toward the Circuit Breaker's 5-call window)
```

Slow / Timeout

```
attempt 1 -> CircuitBreaker permits -> Timeout waits up to 2000ms
-> Payment Service still sleeping (5s) -> TimeoutException
-> not retried (doesn't match retryOnException) -> catch(TimeoutException) -> fallback
-> mechanism=TIMEOUT, attempts=1
```

Circuit OPEN

```
attempt 1 -> CircuitBreaker is OPEN -> lambda never runs, no network call
-> CallNotPermittedException immediately -> not retried -> catch(CallNotPermittedException) -> fallback
-> mechanism=CIRCUIT_BREAKER, attempts=1, paymentServiceCalled=false
```

14. Running the Project

```
# Terminal 1
cd payment-service
mvnw.cmd spring-boot:run      # port 8081

# Terminal 2
cd order-service
mvnw.cmd spring-boot:run      # port 8082

# Terminal 3 (needs Docker Desktop running)
docker compose -p resilience-demo up -d    # Prometheus :9092, Grafana :3030 (admin/admin)
```

Open `ui/index.html` directly in a browser — no build step. Full setup/verification detail and troubleshooting are in `README.md`.